Machine Learning

Notes by Marco Galletti
Lecturer: Stephan Günnemann

Technical University of Munich
Winter Semester 2026/27

Contents

1 k-Nearest Neighbors **2**
1.1 The k-NN algorithm 2
1.2 Choosing k and measuring performance 2
1.3 Distances, scaling and dimensionality 3

1 k-Nearest Neighbors

1.1 The k-NN algorithm

1-NN. Let $\mathcal{D} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^N$ be a training set with features $\boldsymbol{x}_i \in \mathbb{R}^D$ and class labels $y_i \in \{1, \dots, C\}$. The **1-NN** rule fixes a distance measure d (e.g. Euclidean) and gives a new observation the label of its nearest training point; the same rule does regression by copying a real-valued y_i. 1-NN corresponds to the Voronoi tessellation of the training points: each point labels its own cell, and the decision region of a class is the union of the cells of its points. Every single point, a mislabelled one included, owns a cell, so 1-NN generalises poorly.

k-NN. Letting several neighbours vote and taking the majority makes the rule robust against errors in the training set.

Definition 1.1 (k-NN classifier)**.** Let $\mathcal{N}_k(\boldsymbol{x})$ be (the indices of) the k training points nearest to $\boldsymbol{x}$, and let the **indicator variable** $\mathbb{I}(e)$ be 1 if e is true and 0 otherwise. The **k-NN classifier** predicts

$$p(y = c \mid \boldsymbol{x}, k) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\boldsymbol{x})} \mathbb{I}(y_i = c), \qquad \hat{y} = \arg\max_c \; p(y = c \mid \boldsymbol{x}, k),$$

i.e. $\boldsymbol{x}$ gets the mode of its neighbours' labels.

For fixed $\boldsymbol{x}$, $p(\cdot \mid \boldsymbol{x}, k)$ is a probability mass function on the classes (nonnegative, summing to 1): the fraction of each colour in a jar holding the k neighbours' labels, just as a jar with 4 red, 10 green and 6 blue balls gives $p(\text{red}) = 4/20 = 0.2$, $p(\text{green}) = 0.5$, $p(\text{blue}) = 0.3$.

Definition 1.2 (Weighted k-NN, k-NN regression)**.** In **weighted k-NN** each neighbour votes with a weight inversely proportional to its distance:

$$p(y = c \mid \boldsymbol{x}, k) = \frac{1}{Z} \sum_{i \in \mathcal{N}_k(\boldsymbol{x})} \frac{\mathbb{I}(y_i = c)}{d(\boldsymbol{x}, \boldsymbol{x}_i)}, \qquad Z = \sum_{i \in \mathcal{N}_k(\boldsymbol{x})} \frac{1}{d(\boldsymbol{x}, \boldsymbol{x}_i)},$$

and again $\hat{y} = \arg\max_c \; p(y = c \mid \boldsymbol{x}, k)$. In **$k$-NN regression** ($y_i \in \mathbb{R}$) the prediction is the weighted mean of the neighbours' values, with the same Z:

$$\hat{y} = \frac{1}{Z} \sum_{i \in \mathcal{N}_k(\boldsymbol{x})} \frac{y_i}{d(\boldsymbol{x}, \boldsymbol{x}_i)}.$$

Effect of k. With $k = 1$, an isolated training point lying among points of another class (e.g. a mislabelled one) gets its own small island in the decision regions. With $k = 3$ such a point is outvoted by its two other neighbours, so these islands mostly disappear and the decision boundaries become smoother. How to choose k is the topic of the next subsection.

1.2 Choosing k and measuring performance

Train, validation, test. The number of neighbours k is a **hyper-parameter**. The goal is **generalisation**: pick the k that performs best, in a predefined metric such as accuracy, on unseen future data. Future data are not available, so $\mathcal{D}$ is split (fig. 1) into a **learning set** $\mathcal{D}_L$ and a **test set** $\mathcal{D}_t$, and $\mathcal{D}_L$ further into a **training set** $\mathcal{D}_T$ and a **validation set** $\mathcal{D}_V$. **Hyper-parameter tuning** then goes as follows:

1. learn the model on $\mathcal{D}_T$;
2. evaluate it on $\mathcal{D}_V$ for each candidate k and keep the best one;
3. report the final performance on $\mathcal{D}_t$.

In practice one plots the validation accuracy against k and takes the maximiser. More good practice follows in the lecture on decision trees.

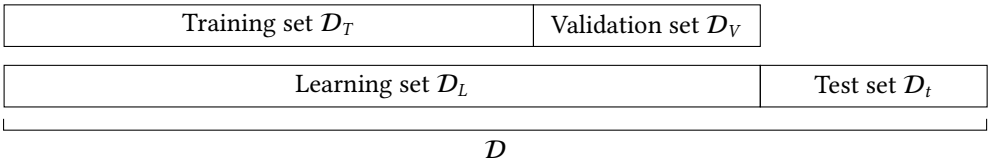


Figure 1: Splitting the dataset $\mathcal{D}$ for hyper-parameter tuning.

Binary classification metrics. For $y \in \{0, 1\}$, the predictions on a labelled set are counted in the **confusion table** (table 1): **true positives** TP and **true negatives** TN are the correct predictions, **false positives** FP and **false negatives** FN the wrong ones.

	True condition	
Predicted	$y = 1$	$y = 0$
$\hat{y} = 1$	TP	FP
$\hat{y} = 0$	FN	TN

Table 1: Confusion table of a binary classifier.

Definition 1.3 (Classification metrics)**.**

$$\text{accuracy:} \quad \text{acc} = \frac{TP + TN}{TP + TN + FP + FN}, \qquad \text{F1 score:} \quad f1 = \frac{2 \cdot \text{prec} \cdot \text{rec}}{\text{prec} + \text{rec}},$$

$$\text{precision:} \quad \text{prec} = \frac{TP}{TP + FP}, \qquad \text{sensitivity/recall:} \quad \text{rec} = \frac{TP}{TP + FN},$$

$$\text{specificity:} \quad \text{tnr} = \frac{TN}{FP + TN}, \qquad \text{false negative rate:} \quad \text{fnr} = \frac{FN}{TP + FN},$$

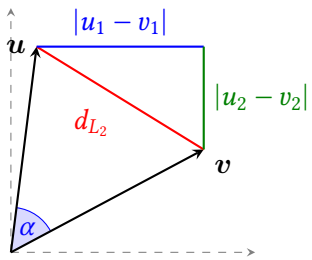
$$\text{false positive rate:} \quad \text{fpr} = \frac{FP}{FP + TN}.$$

Precision and recall trade off: increasing one most often decreases the other. All these numbers must be read with care when the classes are imbalanced.

1.3 Distances, scaling and dimensionality

Distance measures. k-NN works with any distance measure, which makes it highly flexible. For $\boldsymbol{u}, \boldsymbol{v} \in \mathbb{R}^D$ (fig. 2):

- **Euclidean** (L_2) distance $\|\boldsymbol{u} - \boldsymbol{v}\|_2 = \sqrt{\sum_j (u_j - v_j)^2}$;
- L_1 distance $\sum_j |u_j - v_j|$ and L_∞ distance $\max_j |u_j - v_j|$;
- the **angle** α between them, $\cos \alpha = \dfrac{\boldsymbol{u}^T \boldsymbol{v}}{\|\boldsymbol{u}\| \|\boldsymbol{v}\|}$;
- the **Mahalanobis distance** $\sqrt{(\boldsymbol{u} - \boldsymbol{v})^T \boldsymbol{\Sigma}^{-1} (\boldsymbol{u} - \boldsymbol{v})}$, with $\boldsymbol{\Sigma}$ symmetric positive definite;
- Hamming distance, edit distance, and others.



$$d_{L_1} = |u_1 - v_1| + |u_2 - v_2|$$
$$d_{L_2} = \sqrt{(u_1 - v_1)^2 + (u_2 - v_2)^2}$$
$$d_{L_\infty} = \max_j |u_j - v_j| = |u_1 - v_1|$$

Figure 2: L_1, L_2, L_∞ distances and angle α between two vectors of $\mathbb{R}^2$.

Scaling. A distance mixes all features, so features on very different scales distort it. For instance, if in $\mathbb{R}^2$ the feature x_1 is of the order of metres and x_2 of the order of centimetres (say x_1 up to 1000 cm, x_2 below 6 cm), the distance, hence the nearest neighbour, is determined almost only by x_1: the 1-NN decision regions become stripes orthogonal to the x_1-axis and x_2 is effectively ignored. Two remedies:

- **standardisation**: scale each feature j to zero mean and unit variance, $x_{j,\text{std}} = \dfrac{x_j - \mu_j}{\sigma_j}$, with μ_j, σ_j the mean and standard deviation of feature j; this is standard practice in machine learning, since many models are sensitive to differences in scale;
- the Mahalanobis distance with $\boldsymbol{\Sigma} = \text{diag}(\sigma_1^2, \dots, \sigma_D^2)$, which equals the Euclidean distance between the standardised data.

Curse of dimensionality. Draw $N = 20$ samples uniformly from the discrete input space $\{1, \dots, 10\}^D$. For $D = 1$ they cover most or all of the 10 points, for $D = 2$ only about 18% of the 100 points, and for $D = 3$ at most 2% of the 1000 points. Hence in high dimension the nearest neighbour of a new point is typically far away, and to cover the same fraction of the input space (10^D points) N has to grow exponentially with the number of features: this is the **curse of dimensionality**, to keep in mind when using k-NN on high-dimensional data.

Cost. Naive k-NN is expensive: memory and inference are both $O(N)$ (for fixed D), since the whole training set is stored and every query is compared with all training points. Tree-based search structures such as the **k-d tree** give efficient (approximate) nearest-neighbour search, at the price of an additional computation performed only once (building the structure).